



Cypress bij RIVM

- MijnRIVM – vaccinaties
 - cy.session()
 - JSON schema validation
 - Feature flags
 - Accessibility

The image shows two screenshots of the RIVM website. The top screenshot is the 'MijnRIVM - vaccinaties' login page. It features a yellow header with the RIVM logo and navigation links. The main content area lists the types of vaccines covered (Corona, HPV) and provides instructions on how to use the service. There are buttons for 'Inloggen met DigiD' and a help icon. The bottom screenshot is the 'Mijn vaccinaties' page for a user named Aafke Aaltje Aletta Antonz. It shows the user's name, birth date, and selected vaccine programs (COVID and Rijksvaccinatieprogramma). Below this, it lists two vaccination records: one from February 8, 2021, and another from August 4, 2020, both for HPV. Each record includes a table with columns for Naam, Productnaam, and Batchnummer. At the bottom, there is a section for 'Betekenis afkortingen' (Meaning of abbreviations) and a 'Print' button.

MijnRIVM - vaccinaties

Hier vindt u een overzicht van de volgende vaccinaties:

- Coronaprik(ken) van uzelf of uw kind
- HPV-vaccinaties die u hebt gekregen vanuit het Rijksvaccinatieprogramma (alleen voor 16- en 17-jarigen)

Let op:

- Dit overzicht is geen officieel vaccinatiebewijs.
- Het kan enkele dagen duren voordat uw vaccinatie in het overzicht staat.
- In het overzicht staan geen reizigersvaccinaties of vaccinaties die voor werk of opleiding zijn gegeven. Neem hiervoor contact op met de arts of organisatie die u de vaccinatie(s) heeft gegeven.

Ik wil naar mijn vaccinaties gaan

[Inloggen met DigiD](#) [?](#)

TEST - MijnRIVM

Mijn vaccinaties

Naam
Aafke Aaltje Aletta Antonz

Geboortedatum
15-10-2007

☒ COVID ☒ Rijksvaccinatieprogramma

Deze vaccinaties zijn gegeven

8 februari 2021			
	Naam	Productnaam	Batchnummer
	HPV	Cervarix®2	AHPVA395AP

4 augustus 2020			
	Naam	Productnaam	Batchnummer
	HPV	Cervarix®2	AHPVA395AO

Betekenis afkortingen

[COVID](#) ➤ Coronavirus
[HPV](#) ➤ Humaan papillomavirus

Print

Preparations

Technical requirements:

- a. Administrator permissions
- b. Node (18.X --> LTS)
- c. Git (optional)

#0 Setup local cypress project (1)

1.Clone skeleton project from:

github.com/EwaldVerhoeven/tc-cypress-workshop



```
~/repos$ clone  
git@github.com:EwaldVerhoeven/tc-  
cypress-workshop.git
```

2.Open the project in VSCode (or any IDE)



```
~/repos$ code tc-cypress-workshop
```

#0 Setup local cypress project (2)

1. Move into the project (root) folder



```
~/repos$ cd tc-cypress-workshop
```

2. Install dependencies



```
~/repos/tc-cypress-workshop$ npm i
```

EXPLORER

...

▼ TC-CYPRESS-WORKSHOP



> node_modules

> tests

📁 .gitignore

TS cypress.config.ts

{ } cypress.env.EXAMPLE.json

{ } package-lock.json

{ } package.json

📖 README.md

TS tsconfig.json

#0 Setup local cypress project (3)

1. Add 'E2E' object to config

```
import { defineConfig } from "cypress";

export default defineConfig({
  chromeWebSecurity: false,

  e2e: {
    supportFile: "tests/support/e2e.ts",
    specPattern: "tests/e2e/*.cy.{js,jsx,ts,tsx}",
    baseUrl: "https://test-rpg.vercel.app",
    watchForFileChanges: false,
    screenshotsFolder: "tests/screenshots",
    videosFolder: "tests/videos",
    fixturesFolder: "tests/fixtures",

    viewportHeight: 960, // like macbook-16
    viewportWidth: 1650, // bigger than macbook-16. To avoid
    horizontal scrolling
  },
});
```

Locators & selectors

Selector	Recommended	Notes
<code>cy.get('button').click()</code>	⚠ Never	Worst - too generic, no context.
<code>cy.get('.btn.btn-large').click()</code>	⚠ Never	Bad. Coupled to styling. Highly subject to change.
<code>cy.get('#main').click()</code>	⚠ Sparingly	Better. But still coupled to styling or JS event listeners.
<code>cy.get('[name="submission"]').click()</code>	⚠ Sparingly	Coupled to the <code>name</code> attribute which has HTML semantics.
<code>cy.contains('Submit').click()</code>	✅ Depends	Much better. But still coupled to text content that may change.
<code>cy.get('[data-cy="submit"]').click()</code>	✅ Always	Best. Isolated from all changes.

Return vs Yield

Cypress commands YIELD (not RETURN!!) subjects



```
// THIS WILL NOT WORK  
const button = cy.get("button")  
  
button.click()
```

SUT

- <https://test-rpg.vercel.app/>
- <https://github.com/marcelblijleven/testrpg> (source code)

#1 Write your first test (1)

- Add: (test)file to your e2e folder called *rpgManualLogin.cy.ts*



```
describe("My first test", () => {  
  it("Successful manual login", function() {  
    // testcode here  
  })  
})
```

Running your tests

- *Command line (HEADLESS)*

```
npm  Yarn  pnpm  
npx cypress run
```

- *Cypress test runner (HEADED) --> DEMO!*

```
npm  Yarn  pnpm  
npx cypress open
```

#1 Write your first test (2)

- Automate the steps below

Hint: api's you'll need are `cy.visit()` / `cy.get()` / `.click()` / `.type()`

Step	actions
1	Click on the 'login' button
2	Enter a valid email
3	Enter a password
4	Press 'enter' key OR Click on 'login' button in modal

#1 Write your first test (3)

- **Add: *assertions*** (according to 'results' column below) for **step 1 and 4**

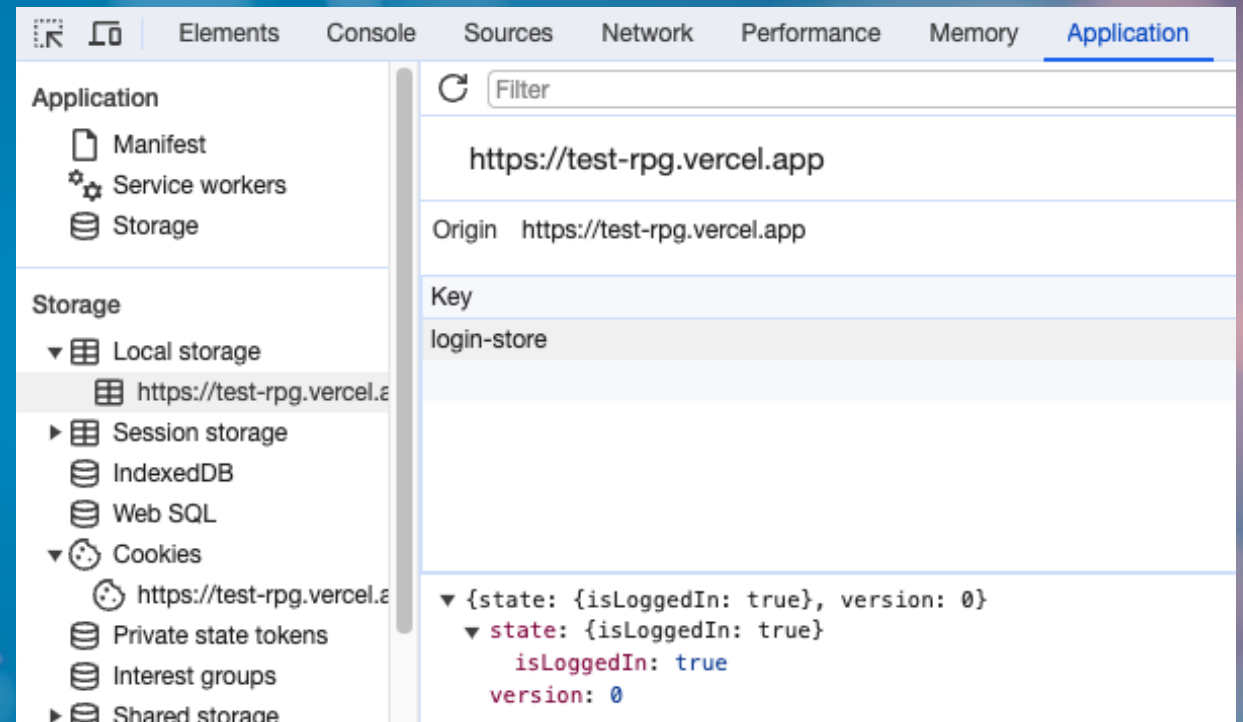
Step	actions	result
1	Click on the 'login' button	Modal with form appears
2	Enter a valid email	No validation messages
3	Enter a password	
4	Press 'enter' key OR Click on 'login' button in modal	Modal exits and button (step 1) now displays 'Logout'

Hint: api you'll need **.should()**

#1 Write your first test (4)

- **Add:** Assertions to check that you are (also technically) logged in

Hint:
*What is in your browsers
local storage?*

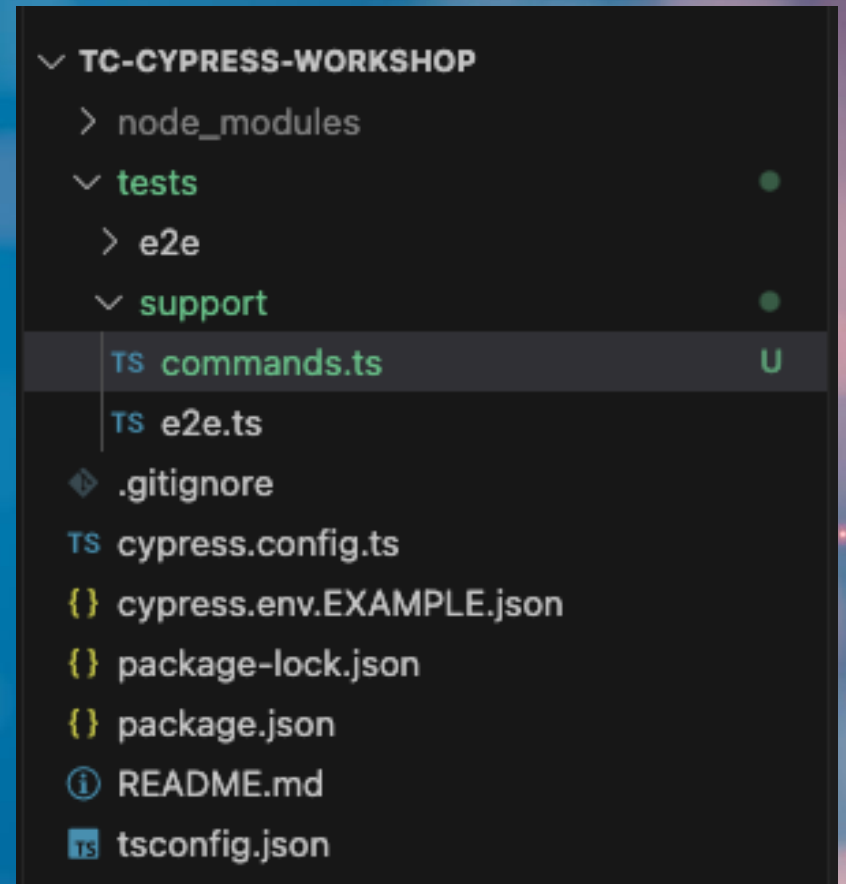


cy.session()

- Used to **persist data** across multiple tests or within a test.
- This allows certain actions, like **logging in**, **cache**, or **cookies**, to be retained, so they don't have to be repeated for every test.
- Makes tests **faster** and **more efficient**, as repeated actions across multiple tests can be avoided.

#2 Custom commands (1)

- Add a file called 'commands.ts' in *./tests/support*



#2 Custom commands (2)

- Write a custom command named 'login' to automate/re-use the 'manual' login procedure (as done in previous exercise).\

```
//commands.ts
```

```
Cypress.Commands.add("login", (username, password) => {  
  //automation code here  
});
```

- Add your custom command to the Cypress namespace and import the commands module in your support file

#2 Custom commands (3)

- Add your custom command to the Cypress namespace (1) and import the commands module in your support file (2)



```
//e2e.ts
```

```
import "../commands.ts";
```

#2 Custom commands (4)

- Refactor your test (*rpgManualLogin.cy.ts*) by using your 'login' custom command --> `cy.login(usr, pwd)`
- **BONUS:** Make use of the `.session()` api in your 'login' command. This allows you restore session data

#3 Hooks and fixtures

1. Automate the following script in a new test file called '*playTheGame.cy.ts*'

Step	Actions	Result
0	Navigate to baseUrl & Login	Browser at baseUrl
1	Click on <button> with text: "Click here to play"	Browser at 'play'
2	Enter Character name	
3	Select option from "Build"	Desired option is selected
4	Click on <button> with text: "Start!"	
5	Click (5 times) on the <button> with tekst: " Click me ... times"	Blue text appears that says 'Great job! You levelled up'
6	Upload a file by clicking 'choose file'	Blue text appears that says 'File selected, level up!'
7	Enter 'Lorem Ipsum' in the 'Type it' field	Blue text appears that says 'Dolar sit amet'
8	Slide the slider all the way to the right	Blue text appears that says 'Slid to the next level!'

Hooks syntax



```
before(function () {  
  cy.visit('/');  
})
```

```
it('test #1', function() {  
  // The rest of your test  
})
```

#3 Hooks and fixtures (1)

- Refactor the test by putting the login action in a 'hook'.
- Brain teaser:
What hook is the best '*type of hook*' to perform teardown and cleanup stuff?

#3 Hooks and fixtures (2)

- Add a 'fixtures' directory (under ./tests)
- Add a file (in the fixtures directory) called '*testdata.json*' and add an object with the following key/value pairs

```
{  
  "role_1": {  
    "name": "Henk",  
    "build": "Mage"  
  }  
}
```

#3 Hooks and fixtures (3)

- Add / load the fixture (*using cy.fixture()*) in the appropriate hook
- Refactor your test to use the fixture instead of 'hardcoded' data.

#4 Network requests



```
cy.request( )
```



```
cy.intercept( )
```

#4 Network requests (1)

- Add a POST request to the available endpoint (using `.request()`). Put it in a after/before hook.
(checkout: <https://test-rpg.vercel.app/api> for endpoints)
- **BONUS:**
use the `.then()` method to yield the response object and assert the expected status code (200) and/or (console.)log the response object.

#4 Network requests (2)

- Spy on a GET request (sent by the front-end) to the '/api/builds' endpoint and assert the expected status code (200).
- *Hint: you'll need (at least) 4 different methods -> .intercept() being one of them...*

#4 Network requests (3)

- Alter the response body (i.e. 'Mock') of the same call to the '/api/builds' to set all 'skills levels' to there maximum value.
- See the effect of your mocked data in the app

Demo JSON schema validation

- JSON schema: Create a schema that defines the expected structure and data types for the JSON response (e.g., required fields, types like string, number, etc.).
- Libraries: Cypress doesn't natively support JSON schema validation, but you can use third-party libraries like ajv (Another JSON Validator) to perform the validation.
- Validate JSON response: After making an API request in Cypress, use the schema to validate the structure of the JSON response.

#5 Page object model (0)

```
cy.get("a[href='/play']")
  .should("be.visible")
  .click({ force: true });
cy.get("[data-testid='character-card']")
// yields 2 elements
  .first()
  .next()
  .find("input[name='name']")
  .should("be.visible")
  .type("somename");
cy.get("select")
  .should("be.visible")
  .select("mage", { force: true });
cy.get("button")
  .contains("Start!")
  .should("be.visible")
  .click();
```

```
HomePage.clickPlayButton();
PlayPage.enterName(role.name);
PlayPage.selectBuild(role.build);
PlayPage.clickStart();
```

#5 Page object model (1)

1. Add a subfolder 'pages' at ./tests/e2e and add and *index.ts* file

```
▼ TC-CYPRESS-WORKSHOP
  > node_modules
  ▼ tests
    ▼ e2e
      ▼ pages
        TS index.ts
      placeholder.gitignore
```

#5 Page object model (2)

- Add your first pageObject file and name it '*homePage.po.ts*'

BONUS:

Add and define a base class in '*base.po.ts*' for other page objects (classes) to inherit from



```
export default class BasePageObject {  
  readonly page = {  
    title: () => cy.title(),  
  };  
}
```

#5 Page object model (3)

- Define the page object (HomePage) and an attribute for all the buttons on the page.
- Add the 'playButton' plus locator/selector code (*hint: use arrow function*)

```
import BasePageObject from "../base.po";

class HomePage extends BasePageObject {
  readonly button = {
    playButton: () => cy.locator(selector),
  };
}

export default new HomePage();
```

#5 Page object model (3)

- Add methods to the class that represents 'Actions' on the html elements. PAY ATTENTION TO READABLE NAMING!!!

```
import BasePageObject from "../base.po";

class HomePage extends BasePageObject {
  readonly button = {
    playButton: () => cy.get("a[href='/play']"),
  };

  clickPlayButton(): void {
    this.button.playButton().//assertions and actions
  }
}

export default new HomePage();
```

#5 Page object model (3)

- Add your (exported) pageobject to the index.ts file and then import the pageobject(s) in your test files (*.cy.ts).

```
//index.ts
export { default as HomePage } from "../homePage.po";
```

```
//mySpec.cy.ts
import { HomePage } from "../pages";

describe("Test", () => {
  // ...
})
```

- Refactor the test(s) to use the pageobject methods in your test (*.cy.ts)

Feature flags

We can retrieve the feature flags from the application config (request to /init-config) and determine if a set of tests or specific test need to run.

```
import FeatureFlag from '../support/feature_flag'

// When there is a featureflag: FEATURE_FLAG_CONTACT:false then testcase 3 and 4 will run.

FeatureFlag('FEATURE_FLAG_CONTACT', true, () => {
  describe('Testcases', () => {
    it('Testcase 1', () => {
      cy.log('1')
    })
  })
})

describe('Testcases', () => {
  FeatureFlag('FEATURE_FLAG_CONTACT', true, () => {
    it('Testcase 2', () => {
      cy.log('2')
    })
  })

  it('Testcase 3', () => {
    cy.log('3')
  })
})

FeatureFlag('FEATURE_FLAG_CONTACT', false, () => {
  describe('Testcases', () => {
    it('Testcase 4', () => {
      cy.log('4')
    })
  })
})
})
```

Tekton Pipelines - OpenShift

- Kubernetes-native: Tekton is specifically built for Kubernetes environments, and OpenShift provides seamless integration, resulting in a scalable and robust CI/CD solution.
- Container-based: Each step within a pipeline runs in its own container, ensuring that the processes are fully isolated and consistent.
- Modular: Tekton offers flexibility by composing pipelines from multiple modular tasks, which can be reused across different workflows.
- OpenShift integration: The OpenShift Pipelines Operator simplifies the installation and management of Tekton, and the UI provides a user-friendly way to manage pipelines.
- CI/CD automation: By combining Triggers, Webhook integration, and automatic scaling, you can automate entire CI/CD processes from source code to production deployment.



OTA-INT

Administrator

Home

Operators

Workloads

Networking

Storage

Builds

Pipelines

Pipelines

Tasks

Triggers

User Management

Administration

Project: rivm-clientportaaltools-co

Pipelines

PipelinesPipelineRunsRepositories

Filter

Name

Search by name...

Name	Last run	Task status	Last run status	Last run time
PL clientportaal-build-deploy	PLR clientportaal-build-deploy-g9xvf9		✔ Succeeded	🕒 5 sep 2024 10:16
PL clientportaal-ci-cd	-	-	-	-
PL cypress	-	-	-	-
PL cypress-acc	-	-	-	-
PL cypress-tst-accessibility	-	-	-	-
PL cypress-tst-nostub	-	-	-	-
PL hielprik-stub	-	-	-	-
PL linkchecker-build-deploy	-	-	-	-
PL linkchecker-run	-	-	-	-
PL psie-api-stub	-	-	-	-

⚙ Administrator ▾

Home >

Operators >

Workloads >

Networking >

Storage >

Builds >

Pipelines ▾

Pipelines

Tasks

Triggers

User Management >

Administration >

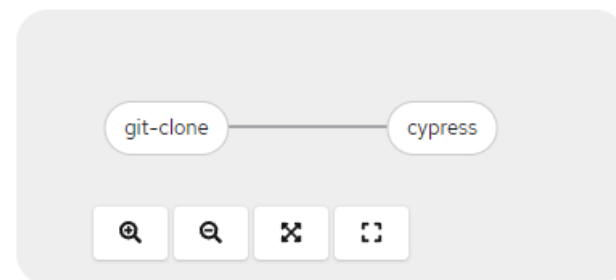
Project: rvm-clientportaaltools-co ▾

[Pipelines](#) > Pipeline details

PL cypress

[Details](#) [YAML](#) [PipelineRuns](#) [Parameters](#) [Metrics](#)

Pipeline details



Name

cypress

Namespace

NS [rvm-clientportaaltools-co](#)

Labels

[Edit](#) 

egress=shared

Annotations

[1 annotation](#) 

Tasks

CT [git-clone](#)

T [cypress](#)

Workspaces

[workspace](#)

Axe plugin in Cypress

The Axe plugin in Cypress is a powerful and easy way to add accessibility checks to your end-to-end tests. The Axe plugin covers approximately 50-57% of WCAG 2.1 guidelines automatically. Axe focuses on rules that can be tested automatically, such as color contrast, semantic HTML structure, and correct ARIA attributes.

However, there are several WCAG criteria that cannot be automatically detected as they require human evaluation. These include:

- Assessing the meaning and context of content.
- Determining if media (such as images or videos) contain correct alternative text in the appropriate context.
- Interactions and keyboard navigation, where the user experience needs to be analyzed.

```
Clienten.Test > cypress > integration > features > Accessibility > TS Accessibility.ts > ...
1  // api
2  import { ClientApi } from '../../api/ClientApi'
3  import { VaccinatiesApi } from '../../api/VaccinatiesApi'
4
5  // api
6  const clientApi = new ClientApi()
7  const vaccinatiesApi = new VaccinatiesApi()
8
9  describe('Accessibility checks', () => {
10     const pages = [
11         '/mijn-gegevens/mijn-vaccinaties',
12         '/mijn-gegevens/planning',
13         '/mijn-gegevens/privacy',
14         '/mijn-gegevens/inzage-dossier',
15         '/vragen',
16         '/contact',
17         '/uitschrijven',
18         '/cookies',
19         '/toegankelijkheid',
20         '/copyright',
21     ]
22
23     before(() => {
24         vaccinatiesApi.intercept()
25         clientApi.intercept()
26         cy.login('user1')
27         cy.wait(['@/vaccinaties', '@/client'])
28     })
29
30     pages.forEach((page) => {
31         it(`Accessibility check for ${page}`, () => {
32             cy.visit(page)
33             cy.injectAxe()
34             cy.configureAxe({
35                 rules: [
36                     {
37                         id: 'aria-progressbar-name',
38                         enabled: false,
39                     },
40                 ],
41             })
42             cy.checkA11y(null, { includedImpacts: ['critical', 'serious'] })
43         })
44     })
45 })
46
```